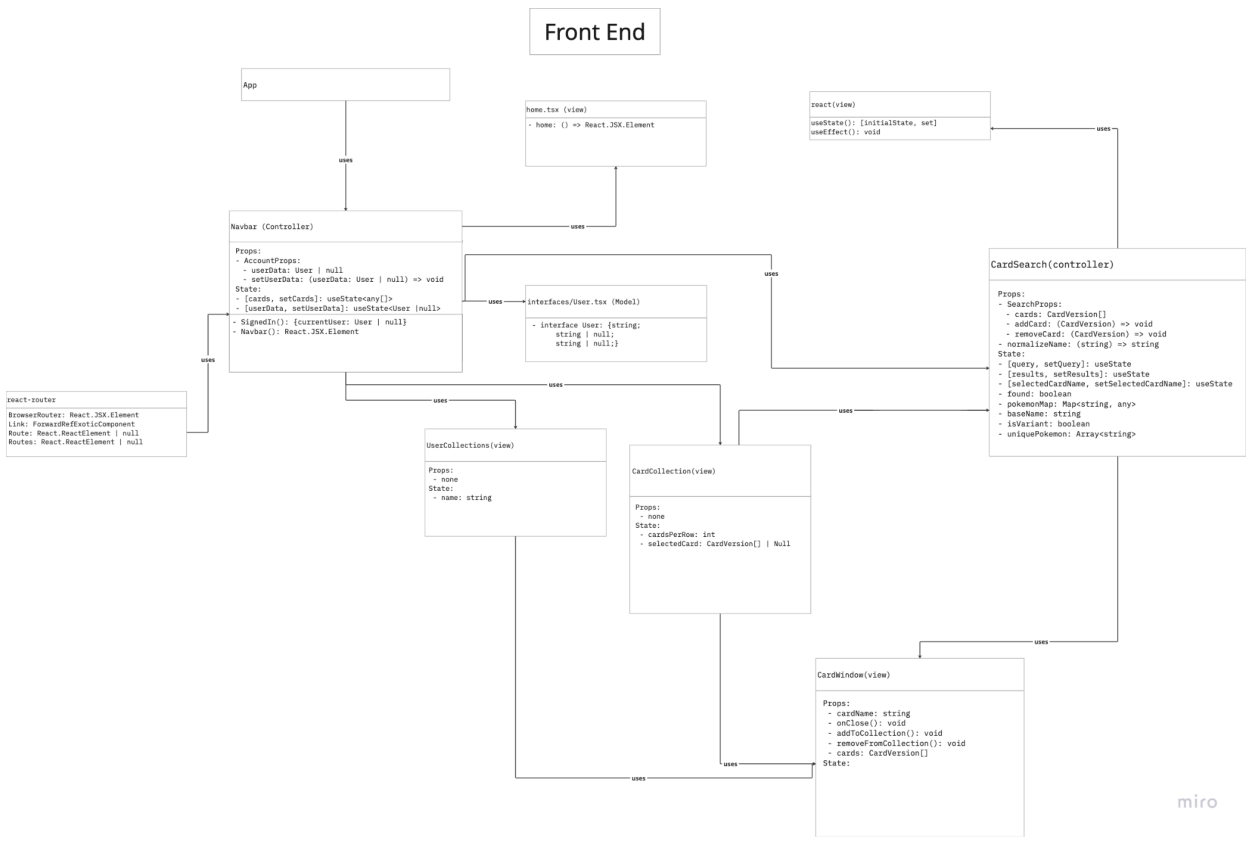
App Overview:

- The **App** component is the index's landing point, and contains only a **Navbar**.
- The primary practical component is the **Navbar**. It contains all pages of the app and routes/links to them all, via React Router.
    - HomePage (**home** component):
        - No purpose as of D4, but will be restructured to display other users' collections in D5.
    - SearchPage (**CardSearch** component):
        - User can search for cards to add to their master and/or specific collections.
    - CollectionsPage (**CardCollection** component):
        - User can view all cards that they have added to their master collection.
    - (Backend) AccountPage (**AccountPage** component):
        - User can log in to their account or sign up if they do not have one.
        - User can also change their display name and email address.

[Miro board link]

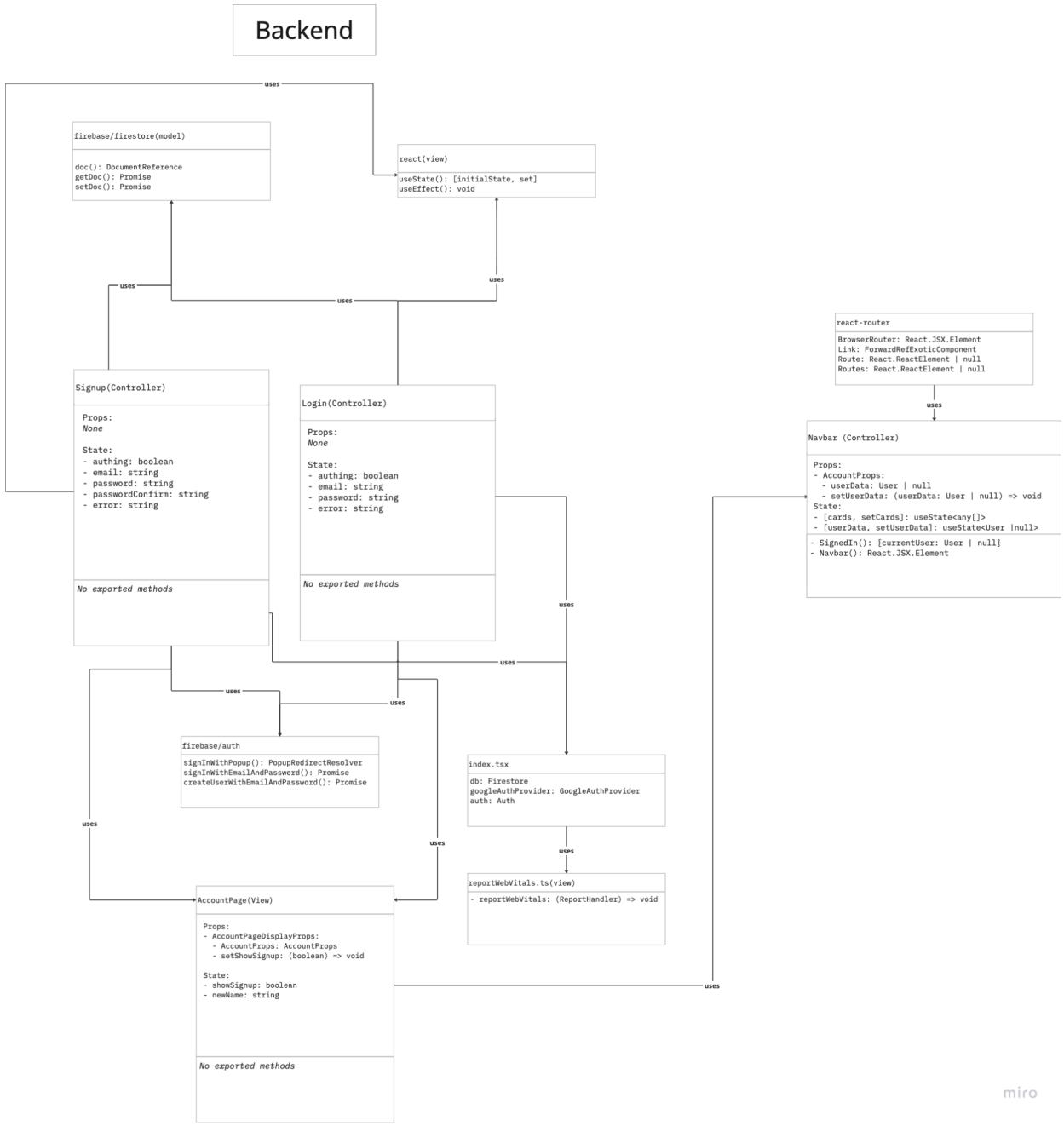


The Frontend diagram shows the interconnections between the different components that the User will interact with while using DieCard. All page components are React

components that utilize the Model-View-Controller organization structure for an intuitive design. The Navbar is the guide for the user to access the most influential parts of the app's State (namely, Cards and UserData). These state properties are passed into the necessary components, such as CardSearch and AccountPage (explained further in Backend).

The CardSearch page gives the user access to the Card API and Search components. When a user searches for a card, it grabs the necessary information from the TCGdex API and feeds it into the CardWindow component. Once here, the user can select information in relation to the card they chose, and is capable of adding this information to their collections

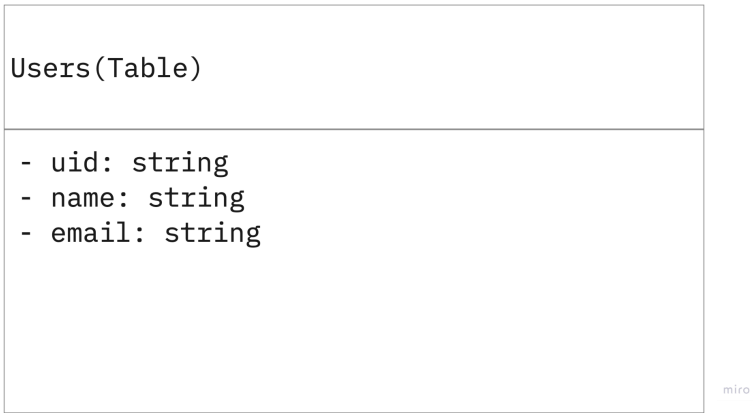
Design Pattern:
The Frontend acts as part of the View-Model-Controller design pattern, representing the Controller and View. The Navbar interface is an access point for the user to reach the sub-interfaces such as CardSearch, CardCollection, and UserCollection. This makes it easier for the user to traverse the different information, while the navigation stays in one location and is ready for the user to interact with something else.



The Backend diagram expands and provides context for the Frontend. As with the Frontend, the Navbar provides the User with a practical way to interact with the database and the application's backend. This is through the AccountPage component (which, as its name suggests, manages the account page). The AccountPage contains the Controlling components, Signup and Login. These interact with the user's account and its place in the database. Both Signup and Login interact with Firestore in Firebase, managed through Firebase's React hooks (particularly, get/setDoc to read/write the user's account information).

Signup and Login contain nearly identical state; both use an authenticating boolean, email, password, and an error message. Signup differs, requiring the user to confirm their password, should they sign up with a standard email address and alphanumeric password. The application handles account verification by using Firebase's "auth" hooks. It's createUserWithEmailAndPassword() and signInWithEmailAndPassword() return Promises that ensure the validity of the user's email and password. The method that handles Google account credentials, signInWithPopup(), functions similarly, instead returning a PopupRedirectResolver that passes authentication to Google's services.

DataBase Tables



As of Deliverable 4, the Database tables are simple and contain a small amount of information about a user's account. The UUID that stores the reference to the user's document is the same as the user's UID stored in the "uid" variable in the document. Alongside "uid", the user's display name ("name") and registered email address ("email") are stored as strings. The Backend's AccountPage reads and writes to these latter two variables when displaying the user's name and providing a way to change the display name and email.